

TEMA 3. PROGRAMACIÓN COMUNICACIONES EN RED. INTRODUCCIÓN.

Hasta ahora hemos visto como varias aplicaciones pueden colaborar entre sí para realizar una tarea de forma conjunta (multiproceso) o bien cómo un mismo programa puede dividir una tarea en partes que se ejecuten de forma concurrente y simultánea (multihilo). Todo esto ocurre dentro de una máquina, bien sea en monoprocesador o multiprocesador, controlados por un mismo SO y compartiendo habitualmente parte de la memoria y de la E/S.

En este tema vamos a ir un paso más allá, vamos a crear aplicaciones que funcionen en entornos distribuidos. Volvemos a tener múltiples procesos en ejecución, pero a diferencia de lo que vimos en el tema 1, en el que los procesos tenían una relación padre-hijo (lanzador-lanzado), ahora los procesos se van a ejecutar en sistemas independientes y se comunicarán a través de la red usando protocolos de comunicación.

Podemos encontrar básicamente dos modelos de sistemas distribuidos:

- Cliente / Servidor: un proceso, denominado servidor, ofrece servicios a uno o más procesos, denominados clientes.
- Entre iguales (P2P): todos los procesos colaboran de forma similar y con un mismo fin, no existiendo una especialización ni diferenciación entre ellos.

Procesos e Hilos

Para realizar un programa distribuido en el que se pueda realizar una conexión y una comunicación a través de una red de ordenadores no partimos de cero.

La programación en red está fuertemente ligada a la programación multiproceso. Principalmente en la forma de comunicación que ya vimos entre procesos.

Por otro lado, la especialización y el servicio que ofrece un servidor, de forma simultánea a varios clientes, está basada en la división del trabajo en hilos.

Por todo lo comentado, todos los conceptos y conocimientos adquiridos hasta ahora nos sirven de base para avanzar en los contenidos de este tema.

Objetivos

- Conocer el protocolo TCP/IP, las direcciones usadas en cada capa y los protocolos asociados.
- Conocer las clases que permiten trabajar con direcciones y nombres de servidores.
- Aprender las características básicas de los protocolos TCP y UDP.
- Desarrollar aplicaciones básicas que se comuniquen usando el protocolo TCP.
- Desarrollar aplicaciones básicas que se comuniquen usando el protocolo UDP.
- Diseñar y programar protocolos para la comunicación entre aplicaciones distribuidas.
- Coordinar la ejecución de múltiples clientes en servidores multihilo.

1.- Conceptos básicos.

Con la fuerte expansión que ha tenido Internet se ha generalizado la utilización de redes en las empresas y en nuestros hogares. Hoy en día para una empresa es totalmente necesario disponer de una red interna que le permita compartir información, conectarse a Internet o incluso, ofrecer sus servicios en Internet.

Cada día es más frecuente que las empresas utilicen aplicaciones que se comunican por Internet para poder compartir información entre sus empleados.

Por ejemplo, aplicaciones de gestión y facturación que permiten que varias tiendas puedan realizar de forma centralizada la facturación de toda la empresa, gestionar el stock, etc.

A continuación, vamos a ver los conceptos más importantes sobre redes, necesarios para más adelante poder programar nuestras aplicaciones. Para ello, primero veremos una pequeña introducción sobre el modelo TCP/IP, los tipos de conexiones que se pueden realizar, así como los modelos más importantes de comunicaciones.

1.1.- Recordando TCP/IP

En 1969 la agencia ARPA (Advanced Research Projects Agency) del Departamento de Defensa de los Estados Unidos inició un proyecto de interconexión de ordenadores mediante redes telefónicas. Al ser un proyecto desarrollado por militares en plena guerra fría un principio básico de diseño era que la red debía poder resistir la destrucción de parte de su infraestructura (por ejemplo, a causa de un ataque nuclear), de forma que dos nodos cualesquiera pudieran seguir comunicados siempre que hubiera alguna ruta que los uniera. Esto se consiguió en 1972 creando una red de conmutación de paquetes denominada ARPAnet, la primera de este tipo que operó en el mundo. La conmutación de paquetes unida al uso de topologías malladas mediante múltiples líneas punto a punto dió como resultado una red altamente fiable y robusta.

ARPAnet fue creciendo paulatinamente, y pronto se hicieron experimentos utilizando otros medios de transmisión de datos, en particular enlaces por radio y vía satélite; los protocolos existentes tuvieron problemas para interoperar con estas redes, por lo que se diseñó un nuevo conjunto o pila de protocolos, y con ellos una arquitectura.

Este nuevo conjunto se denominó TCP/IP (Transmission Control Protocol/Internet Protocol), nombre que provenía de los dos protocolos más importantes que componían la pila; la nueva arquitectura se llamó sencillamente modelo TCP/IP. A la nueva red, que se creó como consecuencia de la fusión de ARPAnet con las redes basadas en otras tecnologías de transmisión, se la denominó Internet.

La aproximación adoptada por los diseñadores del TCP/IP fue mucho más pragmática que la de los autores del modelo OSI. Mientras que en el caso de OSI se emplearon varios años en definir con sumo cuidado una arquitectura de capas donde la función y servicios de cada una estaban perfectamente definidas, y sólo después se planteó desarrollar los protocolos para cada una de ellas, en el caso de TCP/IP la operación fue a la inversa; primero se especificaron los protocolos, y luego se definió el modelo como una simple descripción de los protocolos ya existentes. Por este motivo el modelo TCP/IP es mucho más simple que el OSI. También por este motivo el modelo OSI se utiliza a menudo para describir otras arquitecturas, como por ejemplo TCP/IP, mientras que el modelo TCP/IP nunca suele emplearse para describir otras arquitecturas que no sean la suya propia.

1.1.1.- Recordando TCP/IP (II).

El modelo TCP/IP tiene sólo cuatro capas:

La capa host-red. Esta capa permite comunicar el ordenador con el medio que conecta el equipo a la red.

La capa de red. Esta capa es el eje de la arquitectura TCP/IP ya que permite que los equipos envíen paquetes en cualquier red y viajen de forma independiente a su destino (que podría estar en una red diferente). Los paquetes pueden llegar incluso en un orden diferente a aquel en que se enviaron, en cuyo caso corresponde a las capas superiores reordenándolos, si se desea la entrega ordenada. La capa de red define un formato de paquete y protocolo oficial llamado IP (Internet Protocol). El trabajo de la capa de red es entregar paquetes IP a su destino. Aquí la consideración más importante es decidir el camino que tienen que seguir los paquetes (encaminamiento), y también evitar la congestión. En esta capa se realiza el direccionamiento lógico o direccionamiento por IP, ya que ésta es la capa encargada de enviar un determinado mensaje a su dirección IP de destino.

La capa de transporte. La capa de transporte permite que los equipos lleven a cabo una conversación. Aquí se definieron dos protocolos de transporte: TCP (Transmission Control Protocol) y UDP (User Datagram Protocol). El protocolo TCP es un protocolo orientado a conexión y fiable, y el protocolo UDP es un protocolo no orientado a conexión y no fiable.

En esta capa además se realiza el direccionamiento por puertos. Gracias a la capa anterior, los paquetes viajan de un equipo origen a un equipo destino. La capa de transporte se encarga de que la información se envíe a la aplicación adecuada (mediante un determinado puerto).

La capa de aplicación. Esta capa engloba las funcionalidades de las capas de sesión, presentación y aplicación del modelo OSI. Incluye todos los protocolos de alto nivel relacionados con las aplicaciones que se utilizan en Internet (por ejemplo, HTTP, FTP, TELNET).

1.2.- Conexiones TCP/UDP.

Como se ha visto anteriormente, la capa de transporte cumple la función de establecer las reglas necesarias para establecer una conexión entre dos dispositivos.

Desde la capa anterior, la capa de red, la información se recibe en forma de paquetes desordenados y la capa de transporte debe ser capaz de manejar dichos paquetes y obtener un único flujo de datos. Recuerde, que la capa de red en la arquitectura TCP/IP no se preocupa del orden de los paquetes ni de los errores, es en esta capa donde se deben cuidar estos detalles.

La capa de transporte del modelo TCP/IP es equivalente a la capa de transporte del modelo OSI, por lo que es el encargado de la transferencia libre de errores de los datos entre el emisor y el receptor, aunque no estén directamente conectados, así como de mantener el flujo de la red. La tarea de este nivel es proporcionar un transporte de datos confiable de la máquina de origen a la máquina destino, independientemente de la red física.

Existen dos tipos de conexiones:

TCP (Transmission Control Protocol). Es un protocolo orientado a la conexión que permite que un flujo de bytes originado en una máquina se entregue sin errores en cualquier máquina destino. Este protocolo fragmenta el flujo entrante de bytes en mensajes y pasa cada uno a la capa de red. En el diseño, el proceso TCP receptor reensambla los mensajes recibidos para formar el flujo de salida. TCP también se encarga del control de flujo para asegurar que un emisor rápido no pueda saturar a un receptor lento con más mensajes de los que pueda gestionar.

UDP (User Datagram Protocol). Es un protocolo sin conexión, para aplicaciones que no necesitan la asignación de secuencia ni el control de flujo TCP y que desean utilizar los suyos propios. Este protocolo también se utiliza para las consultas de petición y respuesta del tipo cliente-servidor, y en aplicaciones en las que la velocidad es más importante que la entrega precisa, como las transmisiones de voz o de vídeo. Uno de sus usos es en la transmisión de audio y vídeo en tiempo real, donde no es posible realizar retransmisiones por los estrictos requisitos de retardo que se tiene en estos casos.

De esta forma, a la hora de programar nuestra aplicación deberemos elegir el protocolo que queremos utilizar según nuestras necesidades: TCP o UDP.

1.3.- Puertos de comunicación

Con la capa de red se consigue que la información vaya de un equipo origen a un equipo destino a través de su dirección IP. Pero para que una aplicación pueda comunicarse con otra aplicación es necesario establecer a qué aplicación se conectará. El método que se emplea es el de definir direcciones de transporte en las que los procesos pueden estar a la escucha de solicitudes de conexión. Estos puntos terminales se llaman puertos.

Aunque muchos de los puertos se asignan de manera arbitraria, ciertos puertos se asignan, por convenio, a ciertas aplicaciones particulares o servicios de carácter universal. De hecho, la IANA (Internet Assigned Numbers Authority) determina, las asignaciones de todos los puertos.

Existen tres rangos de puertos establecidos:

Puertos conocidos [0, 1023]. Son puertos reservados a aplicaciones de uso estándar como: 21 – FTP (File Transfer Protocol), 22 – SSH (Secure SHell), 53 – DNS (Servicio de nombres de dominio), 80 – HTTP (Hypertext Transfer Protocol), etc.

Puertos registrados [1024, 49151]. Estos puertos son asignados por IANA para un servicio específico o aplicaciones. Estos puertos pueden ser utilizados por los usuarios libremente.

Puertos dinámicos [49152, 65535]. Este rango de puertos no puede ser registrado y su uso se establece para conexiones temporales entre aplicaciones.

Cuando se desarrolla una aplicación que utilice un puerto de comunicación, optaremos por utilizar puertos comprendidos entre el rango 1024-49151.

1.4.- Nombres en Internet.

Los equipos informáticos se comunican entre sí mediante una dirección IP como 193.147.0.29. Sin embargo, nosotros preferimos utilizar nombres como www.mec.es porque son más fáciles de recordar y porque ofrecen la flexibilidad de poder cambiar la máquina en la que están alojados (cambiaría entonces la dirección IP) sin necesidad de cambiar las referencias a él.

El sistema de resolución de nombres (DNS) basado en dominios, en el que se dispone de uno o más servidores encargados de resolver los nombres de los equipos pertenecientes a su ámbito, consiguiendo, por un lado, la centralización necesaria para la correcta sincronización de los equipos, un sistema jerárquico que permite una administración focalizada y, también, descentralizada y un mecanismo de resolución eficiente.

A la hora de comunicarse con un equipo, puedes hacerlo directamente a través de su dirección IP o puede poner su entrada DNS (p.ej. servidor.miempresa.com). En el caso de utilizar la entrada DNS el equipo resuelve automáticamente su dirección IP a través del servidor de nombres que utilice en su conexión a Internet.

1.5.- Modelos de comunicaciones.

Modelo Cliente/Servidor

Sin duda alguna, el modelo de comunicación que ha revolucionado los sistemas informáticos es el modelo cliente/servidor. El modelo cliente/servidor está compuesto por un servidor que ofrece una serie de servicios y unos clientes que acceden a dichos servicios a través de la red.

Por ejemplo, el servicio más importante de Internet, el WWW, utiliza el modelo cliente/servidor. Por un lado, tenemos el servidor que alojan las páginas web y por otro lado los clientes que solicitan al servidor una determinada página web.

Modelo Sistema de Información Distribuido

Otro modelo ampliamente utilizado son los Sistemas de Información Distribuidos. Un Sistema de Información Distribuido está compuesto por un conjunto de equipos que interactúan entre sí y pueden trabajar a la vez como cliente y servidor. Desde el punto de vista externo es igual que un sistema cliente/servidor ya que el cliente ve al Sistema de Información Distribuido como una entidad. Internamente, los equipos del Sistema de Información Distribuido interactúan entre sí (actuando como servidores y clientes de forma simultánea) para compartir información, recursos, realizar tareas, etc.)

2.- Sockets TCP.

Los sockets permiten la comunicación entre procesos de diferentes equipos de una red. Un socket, es un punto de información por el cual un proceso puede recibir o enviar información.

Tal y como hemos visto anteriormente, existen dos tipos de protocolos de comunicación: TCP o UDP.

En el protocolo TCP es un protocolo orientado a la conexión que permite que un flujo de bytes originado en una máquina se entregue sin errores en cualquier máquina destino.

Por otro lado, el protocolo UDP, no orientado a conexión, se utiliza para comunicaciones donde se prioriza la velocidad sobre la pérdida de paquetes.

A la hora de crear un socket hay que tener claro el tipo de socket que se quiere crear (TCP o UDP).

En esta sección vamos a aprender a utilizar los sockets TCP y en el siguiente punto veremos los sockets UDP.

Al utilizar sockets TCP, el servidor utiliza un puerto por el que recibe las diferentes peticiones de los clientes. Normalmente, el puerto del servidor es un puerto bajo [1-1023].

Socket TCP - Conexión Cuando el cliente realiza la conexión con el servidor, a partir de ese momento se crea un nuevo socket que será el encargado de permitir el envío y recepción de datos entre el cliente/servidor. El puerto se crea de forma dinámica y se encuentra en el rango 49152 y 65535. De esta forma, el puerto por donde se reciben las conexiones de los clientes queda libre y cada comunicación tiene su propio socket.

TCP: transferencia de datos

El paquete `java.net` de Java proporciona la clase `Socket` que permite la comunicación por red. De forma adicional, `java.net` incluye la clase `ServerSocket`, que permite a un servidor escuchar y recibir las peticiones de los clientes por la red. Y la clase `Socket` permite a un cliente conectarse a un servidor para enviar y recibir información.

2.1.- Servidor.

Los pasos que realiza el servidor para realizar una comunicación son:

Publicar puerto. Se utiliza el comando `ServerSocket` indicando el puerto por donde se van a recibir las conexiones.

Esperar peticiones. En este momento el servidor queda a la espera a que se conecte un cliente. Una vez que se conecte un cliente se crea el socket del cliente por donde se envían y reciben los datos.

Envío y recepción de datos. Para poder recibir/enviar datos es necesario crear un flujo (stream) de entrada y otro de salida. Cuando el servidor recibe una petición, éste la procesa y le envía el resultado al cliente.

Una vez finalizada la comunicación se cierra el socket del

cliente.

Esquema de funcionamiento interno del modelo cliente/servidor

Para publicar el puerto del servidor se utiliza la función `ServerSocket` a la que hay que indicarle el puerto a utilizar. Su estructura es:

```
ServerSocket skServidor = new ServerSocket(Puerto);
```

Una vez publicado el puerto, el servidor utiliza la función `accept()` para esperar la conexión de un cliente.

Una vez que el cliente se conecta, entonces se crea un nuevo socket por donde se van a realizar todas las comunicaciones con el cliente y servidor.

```
Socket sCliente = skServidor.accept();
```

Una vez recibida la petición del cliente el servidor se comunica con el cliente a través de streams de datos que veremos en el siguiente punto.

Finalmente, una vez terminada la comunicación se cierra el socket de la siguiente forma:

```
sCliente.close();
```

2.2.- Cliente.

Los pasos que realiza el cliente para realizar una comunicación son:

Conectarse con el servidor. El cliente utiliza la función `Socket` para indicarse con un determinado servidor a un puerto específico. Una vez realizada la conexión se crea el socket por donde se realizará la comunicación.

Envío y recepción de datos. Para poder recibir/enviar datos es necesario crear un flujo(stream) de entrada y otro de salida.

Una vez finalizada la comunicación se cierra el socket.

Esquema de funcionamiento interno del modelo cliente/servidor.

Para conectarse a un servidor se utiliza la función `Socket` indicando el equipo y el puerto al que desea conectarse. Su sintaxis es:

```
Socket sCliente = new Socket(Host , Puerto);
```

donde `Host` es un string que guarda el nombre o dirección IP del servidor y `Puerto` es una variable del tipo `int` que guarda el puerto.

Si lo prefiere también puede realizar la conexión directamente:

```
Socket sCliente = new Socket("192.168.1.200", 1500);
```

Una vez establecida la comunicación, se crean los streams de entrada y salida para realizar las diferentes comunicaciones entre el cliente y el servidor. En el siguiente apartado veremos la creación de streams.

Finalmente, una vez terminada la comunicación se cierra el socket de la siguiente forma:
`sCliente.close();`

2.3.- Flujo de Entrada y de Salida.

Una vez establecida la conexión entre el cliente y el servidor se inicializa la variable del tipo `Socket` que en el ejemplo se llama `sCliente`. Para poder enviar o recibir datos a través del socket es necesario establecer un stream (flujo) de entrada o de salida según corresponda.

Continuando con el ejemplo anterior, a continuación, se va a establecer un stream de salida llamado `flujo_salida`.

```
OutputStream aux = sCliente.getOutputStream();  
DataOutputStream flujo_salida= new DataOutputStream( aux );
```

o lo que es lo mismo,

```
DataOutputStream flujo_salida= new  
DataOutputStream(sCliente.getOutputStream());
```

A partir de éste momento puede enviar información de la siguiente forma:
`flujo_salida.writeUTF( "Enviar datos" );`

De forma análoga, puede establecer el stream de entrada de la siguiente forma:

```
InputStream aux = sCliente.getInputStream();  
DataInputStream flujo_entrada = new DataInputStream( aux );
```

o lo que es lo mismo,

```
DataInputStream flujo_entrada = new  
DataInputStream(sCliente.getInputStream());
```

A continuación, se muestra una forma cómoda de recibir información:

```
String datos=new String();  
Datos=flujo_entrada.readUTF();
```

3.- Sockets UDP.

En el caso de utilizar sockets UDP no se crea una conexión (como es el caso de socket TCP) y básicamente permite enviar y recibir mensajes a través de una dirección IP y un puerto.

Estos mensajes se gestionan de forma individual y no se garantiza la recepción o envío del mensaje como si ocurre en TCP.

Para utilizar sockets UDP en java tenemos la clase `DatagramSocket` y para recibir o enviar los mensajes se utiliza clase `DatagramPacket`. Cuando se recibe o envía un paquete se hace con la siguiente información: mensaje, longitud del mensaje, equipo y puerto.

3.1.- Receptor.

Forma:

En el caso de querer iniciar el socket en un determinado puerto se realiza de la siguiente

```
DatagramSocket sSocket = new DatagramSocket(puerto);
```

Una vez iniciado el socket ya estamos en disposición de recibir mensajes utilizando la clase

DatagramPacket. Cuando se recibe o envía un paquete se hace con la siguiente información: mensaje, longitud del mensaje, equipo y puerto.

A continuación, se muestra un código de ejemplo para recibir un mensaje:

```
byte [] cadena = new byte[1000] ;  
DatagramPacket mensaje = new DatagramPacket(cadena, cadena.length);  
sSocket.receive(mensaje);
```

Una vez recibido el mensaje puede mostrar su contenido de la siguiente forma:

```
String datos=new String(mensaje.getData(),0,mensaje.getLength());  
System.out.println("\tMensaje Recibido: " +datos);
```

Finalmente, una vez terminado el programa cerramos el socket:

```
sSocket.close();
```

3.2.- Emisor.

Por otro lado, para realizar una aplicación emisora de mensajes UDP debe inicializar primero la estructura DatagramSocket.

```
DatagramSocket sSocket = new DatagramSocket();
```

Ahora debe crear el mensaje del tipo DatagramPacket al que debe indicar:

Mensaje a enviar.

Longitud del mensaje.

Equipo al que se le envía el mensaje.

Puerto destino.

A continuación, se muestra un ejemplo para crear un mensaje:

```
DatagramPacket mensaje = new DatagramPacket(mensaje,longitud_mensaje, Equipo, Puerto);
```

Para obtener la dirección del equipo al que se le envía el mensaje a través de su nombre se utiliza la función getByName de la clase InetAddress de la siguiente forma:

```
InetAddress Equipo = InetAddress.getByName("localhost");
```

Una vez creado el mensaje lo enviamos con la función send():

```
sSocket.send(mensaje);
```

socket:

Finalmente, una vez terminado el programa cerramos el `sSocket.close()`;

